PROJECT-BASED LEARNING

PHASE-II EVALUATION

JalRakshak

Intelligent Water Resource Allocation and Scheduling System

Team: HydroLab — Team ID: OSDBMS-V-2026-T111

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun
Academic Session 2026–27

Team & Mentor Details

Team Information

Team Name: HydroLab
Team ID: OSDBMS-V-2026-T111
Semester: 5th
Section: DS1, DS2, ML2, CYBER
Domain: OS, DBMS & Data Science

Team Members

1. Aditya Gupta – 240211284 **(Lead)**
2. Rashi – 240221622
3. Ritesh Chaudhary – 240211067
4. Ritesh Rawat – 24021276

Mentor

Dr. Parmod Mehra

Interactions completed: 8

Phase-I Recap & Feedback Incorporated

Phase-I Status

JalRakshak treats each water request as a process and the supply pipeline as the CPU they compete for. Phase-I built the full pipeline with the FCFS rule. Phase-II (now v2.0) added three more rules, a zone supply timetable, forecasting, a dashboard and security features.

Issues Addressed in Phase-II

- FCFS is unfair: serves by arrival, not by need
- Two admins could allocate the same water twice
- Priorities were set by habit, with no record

Changes Incorporated

- Priority, Round Robin and Water-aware rules added
- Reservoir row lock in one transaction (80,000 vs 160,000 L demo)
- Approved water policy (maker-checker) and decision records

Problem Statement & Finalized Objectives

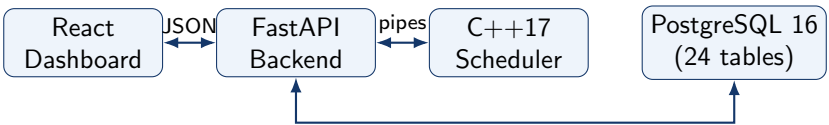
Problem Statement

When available water is less than demand, utilities must decide who gets how much and when, and be able to justify it. Simple rules ignore importance or starve small users, and decisions are rarely compared, approved, recorded or protected from concurrent errors. Seed test case: 8 requests of 80,000 L against a 50,000 L reservoir.

Finalized Objectives

1. Implement FCFS, Priority, Round Robin and Water-aware on one engine and measure them
2. Guarantee no double allocation under concurrency
3. Forecast demand and flag possible leaks
4. Plan zone-wise supply hours with tankers; policy-driven, secure, auditable decisions (SDG 6)

Updated System Architecture / Workflow



backend locks reservoir, runs scheduler as child process, saves everything in one transaction

Major Components

- C++ scheduler: 4 rules, manual mode, timetable
- FastAPI: 95 endpoints; ML job writes via database
- PostgreSQL: locks, triggers, migrations

Changes Since Phase-I

- Policy, timetable, sensors, 2FA, privacy added
- Scheduler runs as child process (10 s timeout)
-

Module Design & Functional Requirements

Major Modules

1. Scheduler Engine (C++)
2. Allocation, Policy & Network Services (FastAPI)
3. Database & Safe Allocation (PostgreSQL)
4. ML Pipeline, Dashboard & Security

Functional Requirements

- Compare FCFS / Priority / RR / Water-aware, then apply one
- Lock reservoir; commit plan in one transaction
- Daily zone timetable and tanker orders
- Forecast demand, detect leaks, show dashboard

Module Dependency

The backend reads pending requests under a row lock, sends them to the C++ scheduler, and stores the run and allocations in PostgreSQL. The ML job and sensors talk to the backend only through the database; the dashboard shows it all.

Technology Stack & Development Methodology

Technology Stack

- Programming: C++17, Python 3.12, SQL, JavaScript
- Database: PostgreSQL 16
- Framework: FastAPI, React 19
- ML: pandas, NumPy, scikit-learn
- Tools: Make, Docker Compose, Git / GitHub

Development Methodology

1. Requirement Analysis
2. System Design
3. Module Development
4. Integration
5. Testing & Validation

Implementation of PBL Course Concepts

Course	Concepts Actually Implemented	Project Module
Operating Systems	FCFS, Priority, Round Robin and Water-aware scheduling; wait, utilisation and Jain fairness metrics; starvation prevention; slot scheduling; mutual exclusion and lock ordering; child process with pipes	Scheduler & Backend
DBMS	24 tables (38 FKs, 49 CHECKs, 49 indexes); ACID transactions; row locks (FOR UPDATE) and advisory locks; triggers; migrations; least-privilege role	Database & Backend

Development & Implementation Progress

Module	Status	Work Completed / Planned
PostgreSQL Database	Completed	24 tables, triggers, migrations; seed of 8 requests (80,000 L)
C++ Scheduler	Completed	4 rules, manual mode, zone timetable; 10,425 checks
FastAPI Backend	Completed	95 endpoints; scheduler run as child process
Safe Allocation	Completed	Row lock, one transaction; concurrency demo done
Docker Setup	Completed	Compose; optional HTTPS and backup (not yet run)
Rule Comparison	Completed	600 synthetic scenarios; Water-aware: no request left empty
Data Science Module	Completed	Demand forecast (MAE 5.1%); leak alerts (precision 0.88)
React Dashboard	Completed	Two panels, 11 admin tabs, Hindi user panel
Testing & Documents	In Progress	Windows Docker run, GitHub CI, HTTPS/backup checks

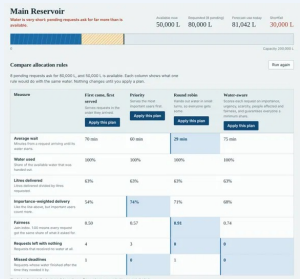
Overall Progress: 8 of 9 deliverables completed, 1 in progress, 0 pending

Working Prototype / Demonstration

Implemented Features

- Four rules compared; one applied by the admin
- Seed case: 80,000 L asked, 50,000 L available
- No double allocation: 1 run with locks, 2 without
- Zone timetable, tankers, sensors, 2FA, Hindi panel

Prototype / Output



Allocate tab: rule comparison (80,000 L asked, 50,000 L available)

Testing, Results & Validation

Test / Module	Expected Result	Observed Result	Status
C++ Scheduler Tests	Rules, metrics and invariants correct	10,425 checks in 33 groups, 0 failures	Pass
Backend API Tests	Lock first, rollback on failure, access rules	77 passed on real PostgreSQL 16	Pass
ML Tests	Forecast and leak detection correct	11 passed	Pass
Concurrency Demo	No water given twice	With locks: 1 run, 80,000 L; without: 2 runs, 160,000 L	Pass
Docker (Windows), HTTPS, Backup, CI	Runs on team laptop and GitHub	Verified	Pass

Results / Validation

5 of 5 test categories passed . The 600-scenario comparison shows Round Robin fairest and Priority best for important users under severe scarcity. All results use synthetic data.

GitHub Repository & Development Activity

Repository Information

- GitHub Link: `https://github.com/AdityaGupta-codes/JalRakshak`
- Branch: main
- README.md: *Updated*
- Contributors: *All Members Added*

Important Commits

- v2.0 release (commit 9b05e7a)
- Four scheduling rules and timetable
- Safe commit with locks and transactions
- ML pipeline and React dashboard

Individual Contribution

Member	Role	Technical Contribution	Commits
Aditya Gupta (Lead)	Water-Aware Scheduler, ML	Water-aware scheduler with weights, backend tests, Docker setup, demand forecast and leak alerts	*XX*
Rashi	C++ Scheduler	FCFS, Priority and Round Robin rules, six measures, scheduler checks	*XX*
Ritesh Chaudhary	FastAPI Backend, ML	Endpoints, safe commit (lock and one transaction), Docker setup, forecasting and leak alerts	*XX*
Ritesh Rawat	Database	PostgreSQL schema, constraints and seed data; concurrency demo	*XX*

Challenges, Remaining Work & Phase-III Roadmap

Challenges Faced

- FCFS is unfair; no rule wins on every measure
- Same water given twice to concurrent requests
- Water-aware has the longest average wait
- No real field data; all results are synthetic

Solutions / Learning

- Four rules compared on one engine
- Row lock, one transaction, lock ordering
- Weights and shares set by approved policy

Phase-III Roadmap

1. Run on Windows Docker; enable GitHub CI
2. Test HTTPS, backup and restore
3. Missing tests: database down, timeout, load
4. Add SJF/EDF rules and timetable rotation
5. Final documentation and demo

Expected Outcomes & References

Expected Final Outcomes

- Fair, safe, explainable water allocation tool
- Zone timetable with tanker orders for short zones
- Forecasting and leak alerts on meter data
- Pilot-ready local system; supports SDG 6

References

1. Silberschatz, Galvin, Gagne – *Operating System Concepts*
2. Elmasri, Navathe – *Fundamentals of Database Systems*
3. PostgreSQL Documentation – transactions & locking
4. FastAPI and React Documentation

Thank You
Questions & Suggestions